ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

BÁO CÁO THỰC HÀNH

LAB 02: LẬP TRÌNH ASSEMBLY CƠ BẢN

Basic Assembly Programming – Lập trình hợp ngữ x86

Nhóm: Nhóm 4
Sinh viên 1: Bùi Thái Sơn – 25521588
Sinh viên 2: Võ Quốc Thịnh – 25521787
Sinh viên 3: Lê Thành Đạt – 25520270

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Chi tiết các bài tập và giải thích mã nguồn 1
  1.1 Bài 2.1: In ra độ dài của chuỗi cho trước (tối đa 9 ký tự) 1
  1.2 Bài 2.2: Tính số dư của tổng 4 số có 2 chữ số khi chia 8 1
  1.3 Bài 2.3: Mã hóa Caesar trên một chuỗi ký tự (Cơ bản + Nâng cao 1 & 2) 3
  1.4 Bài 2.4: Đếm số lượng bội của 10 trong khoảng [x, y] 5
  1.5 Bài 2.5: Kiểm tra tuổi đoàn dựa vào năm sinh (Cơ bản + Nâng cao) 7
  1.6 Bài 2.6: Kiểm tra năm nhuận (Cơ bản + Nâng cao) 10

2 Minh chứng kết quả thực thi 14

1 Chi tiết các bài tập và giải thích mã nguồn

1.1 Bài 2.1: In ra độ dài của chuỗi cho trước (tối đa 9 ký tự)

Yêu cầu đề bài: Cho trước một chuỗi ký tự `msg` được định nghĩa trong `.section .data` (độ dài tối đa 9 ký tự). Hãy viết chương trình hợp ngữ tính độ dài chuỗi (không tính ký tự null kết thúc chuỗi `\0`) và in giá trị này ra màn hình kèm ký tự xuống dòng.

Phân tích giải thuật & Kỹ thuật lập trình:

1. **Tính độ dài tại thời điểm hợp dịch (Assemble-time):** Trong GNU Assembler, ký hiệu `.` đại diện cho con trỏ vị trí hiện tại (location counter). Sau khi khai báo chuỗi `msg: .string "IloveCR7"`, con trỏ vị trí trỏ ngay sau byte null `\0`. Biểu thức `msg_len = . - msg - 1` lấy khoảng cách từ địa chỉ hiện tại trừ đi địa chỉ đầu chuỗi `msg`, rồi trừ tiếp 1 byte null để thu được đúng số ký tự thực tế (8 ký tự).
2. **Định dạng xuất dữ liệu ASCII:** Hàm xuất chuỗi của hệ điều hành yêu cầu dữ liệu là các mã ASCII. Với độ dài chuỗi $L \leq 9$, giá trị số L được cộng thêm `0x30` (tức giá trị ASCII của ký tự `'0'`, tương đương 48) để chuyển đổi thành ký tự chữ số. Ô nhớ `output` gồm 2 bytes được chuẩn bị sẵn: byte thứ nhất chứa mã ASCII độ dài chuỗi (`msg_len + 0x30`), byte thứ hai chứa mã `0x0a` (ký tự xuống dòng `'\n'`).
3. **Thực hiện xuất và thoát:** Nạp địa chỉ `$output` vào `%ecx`, độ dài 2 vào `%edx`, `%ebx = 1` (STDOUT), `%eax = 4` (`sys_write`), sau đó gọi ngắt `int $0x80`. Tiếp theo gọi `sys_exit` (`%eax = 1`) để kết thúc tiến trình an toàn.

Mã nguồn chương trình (`bai2.1.s`):

```
.section .data
msg:
    .string "IloveCR7"
msg_len = . - msg - 1 # Lay dia chi hien tai - dia chi msg - 1 (tru byte null)

output:
    .byte msg_len + 0x30 # Cong 48 (0x30) de chuyen thanh ky tu ASCII
    .byte 0x0a           # Ky tu xuong dong '\n'

.section .text
    .globl _start
_start:
    movl $output, %ecx   # Dia chi buffer can xuat
    movl $0x2, %edx      # Do dai xuat ra: 2 bytes
    movl $0x1, %ebx      # File descriptor: stdout (1)
    movl $0x4, %eax      # System call number: sys_write (4)
    int $0x80            # Goi kernel xuat chuoi

    movl $0x1, %eax      # System call number: sys_exit (1)
    int $0x80            # Thoat chuong trinh
```

1.2 Bài 2.2: Tính số dư của tổng 4 số có 2 chữ số khi chia 8

Yêu cầu đề bài: Nhập lần lượt 4 số nguyên có 2 chữ số ($10 \leq x \leq 99$) từ bàn phím. Tính tổng 4 số đã nhập và in ra màn hình số dư của tổng khi chia cho 8.

Phân tích giải thuật & Kỹ thuật lập trình:

1. **Đọc và chuyển đổi chuỗi số 2 chữ số (`nhapso`):** Mỗi lần người dùng nhập một số có 2 chữ số và nhấn Enter, `sys_read` với độ dài 3 bytes (`%edx = 3`) sẽ đọc được 2 ký tự số và 1 ký tự xuống dòng vào vùng nhớ `number1`.

  - Ký tự hàng chục: Lệnh `movzbl number1, %eax` đọc 1 byte mở rộng không dấu vào `%eax`, trừ đi 48 (`subl $48, %eax`) để lấy giá trị chữ số, rồi nhân 10 bằng `imull $10, %eax`.
  - Ký tự hàng đơn vị: Lệnh `movzbl number1+1, %ebx` đọc byte tiếp theo, trừ đi 48.
  - Tổng số vừa nhập: Cộng `%ebx` vào `%eax`, sau đó cộng dồn vào biến tích lũy `ans` trong bộ nhớ.

2. **Tái sử dụng hàm con:** Lệnh gọi `call nhapso` được thực thi tuần tự 4 lần trong `_start`, giúp mã nguồn súc tích và mạch lạc.

3. **Phép chia lấy số dư (`inkq`):** Lệnh `divl %ecx` trong kiến trúc x86 thực hiện chia số nguyên không dấu 64-bit trong cặp thanh ghi `%edx:%eax` cho toán hạng 32-bit:

$$\text{ans}/8 \implies \texttt{\%eax} = \lfloor \text{ans}/8 \rfloor \quad \text{và} \quad \texttt{\%edx} = \text{ans} \pmod 8$$

Cần xóa sạch thanh ghi `%edx` bằng `xorl %edx, %edx` trước khi chia. Số dư thu được trong `%edx` được cộng thêm 48 để tạo mã ASCII và in ra màn hình cùng thông báo `"Remainder: "`.

Mã nguồn chương trình (`bai2_2.s`):

```
.section .data
nhapvao:
    .string  "Enter a number (2-digit): "
kq:
    .string "Remainder: "
newline:
    .byte 10
ans:
    .int 0

.section .bss
    .lcomm number1, 4

.section .text
    .globl _start

jmp _start

nhapso:
    movl $4, %eax        # sys_write in loi nhac
    movl $1, %ebx
    movl $nhapvao, %ecx
    movl $26, %edx
    int $0x80

    movl $3, %eax        # sys_read doc 3 bytes tu STDIN
    movl $0, %ebx
    movl $number1, %ecx
    movl $3, %edx
    int $0x80

    movzbl number1, %eax    # Doc chu so hang chuc
    subl $48, %eax          # Chuyen thanh gia tri nguyen
    imull $10, %eax         # Nhan voi 10

    movzbl number1+1, %ebx  # Doc chu so hang don vi
    subl $48, %ebx
    addl %ebx, %eax         # Tong gia tri cua so vua nhap

    addl %eax, ans          # Cong don vao bien tong ans
    ret

inkq:
    movl ans, %eax       # Nap tong vao %eax
    xorl %edx, %edx      # Xoa %edx de chuan bi chia 64-bit
    movl $8, %ecx        # So chia = 8
    divl %ecx            # Chia: thuong o %eax, so du o %edx

    movl %edx, ans
    addl $48, ans        # Chuyen so du sang ky tu ASCII

    movl $4, %eax        # In chuoi thong bao "Remainder: "
    movl $1, %ebx
    movl $kq, %ecx
    movl $11, %edx
    int $0x80

    movl $4, %eax        # In ky tu so du
    movl $1, %ebx
    movl $ans, %ecx
    movl $1, %edx
    int $0x80

    movl $4, %eax        # In dau xuong dong
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret

_start:
    call nhapso
    call nhapso
    call nhapso
    call nhapso
    call inkq

    mov $1, %eax         # sys_exit
    int $0x80
```

1.3 Bài 2.3: Mã hóa Caesar trên một chuỗi ký tự (Cơ bản + Nâng cao 1 & 2)

Yêu cầu đề bài:

- **Cơ bản:** Nhập một chuỗi ký tự và số bước dịch n ($0 \leq n \leq 9$), xuất ra chuỗi đã được mã hóa Caesar tương ứng.
- **Nâng cao 1:** Xử lý xoay vòng chuẩn khi ký tự sau khi dịch vượt quá `'z'` hoặc `'Z'` (ví dụ: với $n = 3$, `'z'` chuyển thành `'c'`).
- **Nâng cao 2:** Hỗ trợ đồng thời cả chữ in hoa (`'A'–'Z'`) và chữ in thường (`'a'–'z'`). Tất cả ký tự số, ký tự đặc biệt, khoảng trắng được giữ nguyên không đổi. Chuỗi đầu vào có độ dài tùy chọn tối đa 255 ký tự.

Phân tích giải thuật & Kỹ thuật lập trình:

1. **Nhận chuỗi có độ dài động:** Sử dụng vùng nhớ đệm 256 bytes trong `.bss`. Lời gọi `sys_read` trả về số lượng byte thực tế đã đọc vào `%eax`, giá trị này được lưu vào `input_len` làm giới hạn duyệt chuỗi.
2. **Phân loại ký tự (Filtering):** Duyệt con trỏ `%edx` qua từng byte của chuỗi:
    - Ký tự $<$ ’A’: Nhảy tới `.cipher_loop` (giữ nguyên ký tự đặc biệt, số, dấu câu).
    - Ký tự $\leq$ ’Z’: Là chữ in hoa, đặt mốc cơ sở `%bl = 'A'` và nhảy tới `.shift`.
    - Ký tự $<$ ’a’: Là ký tự đặc biệt nằm giữa bảng mã ASCII (`'['`, `']'`, …), giữ nguyên.

- Ký tự $\le$ ’z’: Là chữ in thường, đặt mốc cơ sở `%bl = 'a'` và nhảy tới `.shift`.
- Ký tự $>$ ’z’: Ký tự đặc biệt, giữ nguyên.

3. **Giải thuật dịch và xoay vòng Modulo 26:**

$$c_{new} = \begin{cases} (c - \text{base} + n) + \text{base} & \text{nếu } (c - \text{base} + n) < 26 \\ (c - \text{base} + n - 26) + \text{base} & \text{nếu } (c - \text{base} + n) \geq 26 \end{cases}$$

Vì $0 \leq n \leq 9$, tổng $(c - \text{base} + n)$ tối đa là $25 + 9 = 34 < 52$. Do đó, chỉ cần so sánh với 26 qua lệnh `cmpb $26, %al`; nếu ≥ 26 thì trừ đi 26 (`subb $26, %al`) là đảm bảo xoay vòng chính xác.

Mã nguồn chương trình (`bai2.3.s`):

```
.section .rodata
enter_string:
    .ascii "Enter a string (<255 chars): "
enter_string_len = .-enter_string

shift_string:
    .ascii "Enter n (0-9): "
shift_string_len = .-shift_string

.section .bss
    .lcomm input_str, 256
    .lcomm input_len, 4
    .lcomm n, 2

.section .text
    .globl _start

printf:
    movl $0x1, %ebx         # stdout
    movl $0x4, %eax         # write
    int $0x80
    ret

read:
    xorl %ebx, %ebx         # stdin
    movl $0x3, %eax         # read
    int $0x80
    ret

cipher:
    movl $0, %ecx
    movl $input_str, %edx
    decl %edx

.cipher_loop:
    cmpl input_len, %ecx
    je .fin
    incl %edx
    incl %ecx
    cmpb $'A', (%edx)
    jb .cipher_loop     # Ky tu ngoai khoang (giu nguyen)
    cmpb $'Z', (%edx)
    movb $'A', %bl
    jbe .shift          # Chu hoa ('A' <= x <= 'Z')

    cmpb $'a', (%edx)
    jb .cipher_loop     # Ky tu dac biet nam giua 'Z' va 'a'
    cmpb $'z', (%edx)
    movb $'a', %bl
    jbe .shift          # Chu thuong ('a' <= x <= 'z')

    jmp .cipher_loop    # Ky tu lon hon 'z'

.fin:
    ret

.shift:
    movb (%edx), %al
    subb %bl, %al       # Dua ve khoang 0 .. 25
    addb n, %al         # Cong buoc dich n
    cmpb $26, %al       # Kiem tra tran 26
    jb .done
    subb $26, %al       # Xoay vong bang cach tru 26

.done:
    addb %bl, %al       # Dua ve lai ma ASCII goc
    movb %al, (%edx)    # Ghi de ky tu da ma hoa vao buffer
    jmp .cipher_loop

_start:
    movl $enter_string, %ecx
    movl $enter_string_len, %edx
    call printf

    movl $input_str, %ecx
    movl $256, %edx
    call read
    movl %eax, input_len

    movl $shift_string, %ecx
    movl $shift_string_len, %edx
    call printf

    movl $n, %ecx
    movl $0x2, %edx
    call read
    subb $0x30, n
    call cipher

    movl $input_str, %ecx
    movl input_len, %edx
    call printf

    movl $1, %eax
    int $0x80
```

1.4 Bài 2.4: Đếm số lượng bội của 10 trong khoảng [x, y]

Yêu cầu đề bài: Nhập từ bàn phím hai số nguyên có 2 chữ số x và y ($10 \leq x \leq y \leq 99$). Đếm số lượng các số chia hết cho 10 nằm trong đoạn đóng $[x, y]$ (tính cả x và y).

Phân tích giải thuật & Kỹ thuật lập trình:

1. **Tối ưu hóa toán học độ phức tạp $O(1)$:** Thay vì sử dụng vòng lặp duyệt từng phần tử từ x đến y, ta áp dụng nguyên lý toán học: số lượng số chia hết cho 10 trong khoảng $[1, N]$ là $\lfloor N/10 \rfloor$. Do đó, số lượng bội của 10 trong đoạn đóng $[x, y]$ được tính trực tiếp bằng:

   $$\text{count} = \left\lfloor \frac{y}{10} \right\rfloor - \left\lfloor \frac{x-1}{10} \right\rfloor$$

   Công thức này có độ phức tạp thời gian $O(1)$, thực hiện đúng đắn cho mọi trường hợp biên khi x hoặc y chính là bội của 10.

2. **Cài đặt trên Assembly:**
   - Đọc chuỗi x, chuyển thành số nguyên: $\text{val}_x = (x[0] -' 0') \times 10 + (x[1] -' 0')$. Giảm đi 1 bằng lệnh `decl %edi`, sau đó chia 10 bằng lệnh `divl %ecx` để thu được thương số $\lfloor (x-1)/10 \rfloor$ lưu vào thanh ghi `%edi`.

  - Đọc chuỗi y, chuyển thành số nguyên, chia 10 để thu được thương số $\lfloor y/10 \rfloor$ lưu vào thanh ghi `%esi`.
  - Hiệu số `subl %edi, %esi` cho ra kết quả cần tìm.

3. **Xuất kết quả trực tiếp qua ngăn xếp (Stack):** Cộng thêm `'0'` vào `%esi` để chuyển sang ký tự ASCII, sau đó đẩy trực tiếp lên đỉnh ngăn xếp bằng `pushl %esi`. Địa chỉ đỉnh ngăn xếp hiện tại (`%esp`) được gán trực tiếp làm con trỏ buffer cho `sys_write` (`movl %esp, %ecx`), không tốn ô nhớ trong `.bss`.

Mã nguồn chương trình (`bai2.4.s`):

```
.section .data
hihi:
    .string "Nhap so: "
string_len = . - hihi
newline:
    .ascii "\n"

.section .bss
    .lcomm x, 3
    .lcomm y, 3

.section .text
.globl _start

_start:
    # 1. Nhap va tinh [(x - 1) / 10]
    movl $4, %eax
    movl $1, %ebx
    movl $hihi, %ecx
    movl $string_len, %edx
    int $0x80

    movl $0, %ebx
    movl $3, %eax
    movl $x, %ecx
    movl $3, %edx
    int $0x80

    movzbl x, %edi
    subl $'0', %edi
    imull $10, %edi

    movzbl x+1, %eax
    subl $'0', %eax
    addl %eax, %edi

    decl %edi            # Tinh x - 1
    movl $10, %ecx
    movl %edi, %eax
    xorl %edx, %edx
    divl %ecx            # Chia 10: thuong nam o %eax
    movl %eax, %edi      # Luu [(x - 1) / 10] vao %edi

    # 2. Nhap va tinh [y / 10]
    movl $4, %eax
    movl $1, %ebx
    movl $hihi, %ecx
    movl $string_len, %edx
    int $0x80

    movl $0, %ebx
    movl $3, %eax
    movl $y, %ecx
    movl $3, %edx
    int $0x80

    movzwl y, %eax
    subb $'0', %al
    subb $'0', %ah

    movzbl %al, %ecx
    imull $10, %ecx
    movzbl %ah, %edx
    addl %edx, %ecx      # y gio nam o %ecx

    movl %ecx, %eax
    movl $10, %ecx
    xorl %edx, %edx
    divl %ecx            # Chia 10: thuong o %eax
    movl %eax, %esi      # Luu [y / 10] vao %esi

    # 3. Tinh hieu so va xuat ket qua
    subl %edi, %esi      # %esi = [y/10] - [(x-1)/10]
    addl $'0', %esi      # Chuyen thanh ky tu ASCII
    pushl %esi           # Day vao dinh ngan xep

    movl $4, %eax
    movl $1, %ebx
    movl %esp, %ecx      # Dung dia chi %esp lam buffer xuat
    movl $1, %edx
    int $0x80

    movl $4, %eax        # Xuat dau xuong dong
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80

    movl $1, %eax        # Thoat
    xorl %ebx, %ebx
    int $0x80
```

1.5 Bài 2.5: Kiểm tra tuổi đoàn dựa vào năm sinh (Cơ bản + Nâng cao)

Yêu cầu đề bài:

- **Cơ bản:** Nhập năm sinh có 4 chữ số từ bàn phím. Nếu tuổi từ 16 đến 30 tuổi thì xuất thông báo `"Thuoc Doan TN"`, ngược lại xuất `"Khong thuoc Doan TN"`.
- **Nâng cao:** Xuất kèm theo số tuổi cụ thể tính được (ví dụ: `"Tuoi: 21"`).

Phân tích giải thuật & Kỹ thuật lập trình:

1. **Chuyển đổi chuỗi 4 chữ số sang số nguyên:** Chuỗi năm sinh gồm 4 ký tự được nhân theo trọng số từng hàng:

   $$\text{nam\_sinh} = (c_0 -' 0') \times 1000 + (c_1 -' 0') \times 100 + (c_2 -' 0') \times 10 + (c_3 -' 0')$$

   Độ tuổi được xác định so với mốc năm hiện tại 2026:

   $$\text{tuoi} = 2026 - \text{nam\_sinh}$$

2. **So sánh khoảng:** So sánh giá trị `tuoi` trong `%edi`:
   - `cmpl $16, %edi`; `jl ngoai_khoang`: Nếu tuổi < 16, rẽ nhánh.
   - `cmpl $30, %edi`; `jg ngoai_khoang`: Nếu tuổi > 30, rẽ nhánh.
   - Nếu thỏa mãn $16 \leq \text{tuoi} \leq 30$: In chuỗi `"Thuoc Doan TN\n"`, sau đó nhảy đến phần nâng cao.

3. **Thuật toán in số nguyên nhiều chữ số qua Ngăn xếp (Stack):** Thủ tục `print` in số nguyên bất kỳ ra màn hình:
    - **Tách chữ số (`chia_loop`):** Lặp chia số cho 10, phần dư được chuyển thành ký tự ASCII (`addb $'0', %dl`) rồi đẩy vào ngăn xếp (`pushl %edx`). Tăng biến đếm chữ số `%esi`. Lặp lại đến khi thương số bằng 0.
    - **In chữ số (`in_loop`):** Rút từng chữ số từ ngăn xếp (`popl %eax`). Do tính chất LIFO, các chữ số được lấy ra đúng thứ tự từ hàng cao nhất đến hàng đơn vị, sau đó xuất lần lượt ra terminal.

Mã nguồn chương trình (`bai2.5.s`):

```
.section .data
hello:
    .ascii "Nhap nam sinh: "
string_len = . - hello

Thuoc:
    .ascii "Thuoc Doan TN\n"
Thuoc_len = . - Thuoc

Khongthuoc:
    .ascii "Khong thuoc Doan TN\n"
Khongthuoc_len = . - Khongthuoc

intuoi:
    .ascii "Tuoi: "
intuoi_len = . - intuoi

newline:
    .ascii "\n"

.section .bss
    .lcomm tuoi, 5
    .lcomm out, 1

.section .text
.globl _start

_start:
    # 1. Nhap nam sinh tu ban phim
    movl $4, %eax
    movl $1, %ebx
    movl $hello, %ecx
    movl $string_len, %edx
    int $0x80

    movl $3, %eax
    movl $0, %ebx
    movl $tuoi, %ecx
    movl $5, %edx
    int $0x80

    # 2. Chuyen 4 ky tu thanh so nguyen trong %edi
    xorl %edi, %edi

    movzbl tuoi, %eax
    subl $'0', %eax
    imull $1000, %eax
    addl %eax, %edi

    movzbl tuoi+1, %eax
    subl $'0', %eax
    imull $100, %eax
    addl %eax, %edi

    movzbl tuoi+2, %eax
    subl $'0', %eax
    imull $10, %eax
    addl %eax, %edi

    movzbl tuoi+3, %eax
    subl $'0', %eax
    addl %eax, %edi

    # 3. Tinh tuoi: 2026 - nam_sinh
    movl $2026, %eax
    subl %edi, %eax
    movl %eax, %edi

    # 4. Kiem tra khoang [16, 30]
    cmpl $16, %edi
    jl ngoai_khoang
    cmpl $30, %edi
    jg ngoai_khoang

    movl $4, %eax
    movl $1, %ebx
    movl $Thuoc, %ecx
    movl $Thuoc_len, %edx
    int $0x80
    jmp nang_cao

ngoai_khoang:
    movl $4, %eax
    movl $1, %ebx
    movl $Khongthuoc, %ecx
    movl $Khongthuoc_len, %edx
    int $0x80

nang_cao:
    movl $4, %eax
    movl $1, %ebx
    movl $intuoi, %ecx
    movl $intuoi_len, %edx
    int $0x80
    call print

done:
    movl $1, %eax
    xorl %ebx, %ebx
    int  $0x80

# Thu tuc print: in so nguyen trong %edi ra man hinh
print:
    movl %edi, %eax
    xorl %esi, %esi

chia_loop:
    xorl %edx, %edx
    movl $10, %ebx
    divl %ebx            # Chia 10 lay so du vao %edx
    addb $'0', %dl       # Chuyen sang ma ASCII
    pushl %edx           # Day vao ngan xep
    incl %esi            # Tang bien dem chu so
    cmpl $0, %eax
    jne chia_loop

in_loop:
    popl %eax            # Rut chu so theo thu tu dung
    movb %al, out
    movl $4, %eax
    movl $1, %ebx
    movl $out, %ecx
    movl $1, %edx
    int $0x80
    decl %esi
    jnz in_loop

    movl $4, %eax        # In ky tu xuong dong
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret
```

1.6 Bài 2.6: Kiểm tra năm nhuận (Cơ bản + Nâng cao)

Yêu cầu đề bài:

- **Cơ bản:** Nhập một năm a có 4 chữ số ($1000 \leq a \leq 9999$). Xuất ra `"Nam nhuan"` nếu là năm nhuận, ngược lại xuất `"Nam khong nhuan"`.
- **Nâng cao:** Tìm và in ra năm nhuận kế tiếp sau năm vừa nhập.

Phân tích giải thuật & Kỹ thuật lập trình:

1. **Quy tắc năm nhuận chuẩn lịch Gregory:** Một năm là năm nhuận khi và chỉ khi:

$$(\text{năm} \pmod{400} == 0) \quad \lor \quad (\text{năm} \pmod{4} == 0 \land \text{năm} \pmod{100} \neq 0)$$

   Chương trình chia thành hai nhánh kiểm tra:

   - Nếu hai chữ số cuối cùng bằng `"00"` (năm tròn thế kỷ, chia hết cho 100): Kiểm tra chia hết cho 400. Nếu chia hết cho 400 thì là năm nhuận; ngược lại không phải.
   - Nếu hai chữ số cuối khác `"00"` (không chia hết cho 100): Chỉ cần kiểm tra hai chữ số cuối chia hết cho 4 hay không.

2. **Tìm năm nhuận kế tiếp (`namnhuantieptheo`):**

   - Tăng năm lên 1 (`addl $1, ans`).
   - Kiểm tra: Nếu chia hết cho 4 và không chia hết cho 100 $\Longrightarrow$ là năm nhuận (`okroido`). Nếu chia hết cho 100 và chia hết cho 400 $\Longrightarrow$ là năm nhuận (`okroido`). Ngược lại, lặp tiếp bước tăng năm.

3. **Kỹ thuật ghi ngược chuỗi số vào bộ đệm:** Con trỏ `%esi` trỏ vào cuối mảng `buffer_num + 10`, thực hiện chia 10 lấy phần dư, lưu ký tự từ phải sang trái qua lệnh `decl %esi; movb %dl, (%esi)`. Độ dài chuỗi được tính bằng `buffer_num + 10 - %esi`, sau đó xuất ra terminal.

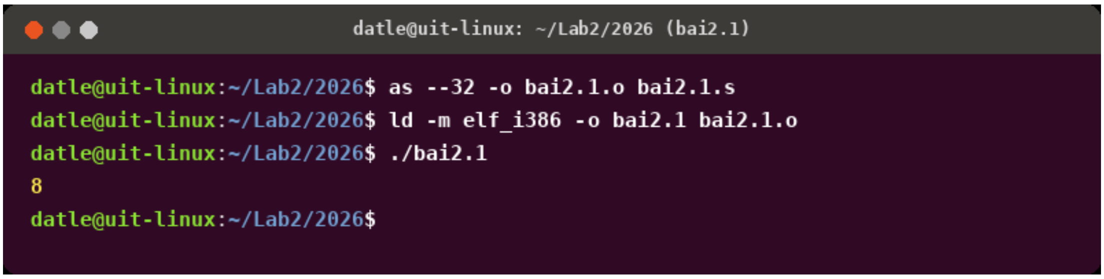
Mã nguồn chương trình (`bai2_6.s`):

```
.section .data
nhapvao:
    .string  "Enter a year (4-digit): "
kqnamnhuan:
    .string "Nam nhuan\n"
kqnamkhongnhuan:
    .string "Nam khong nhuan\n"
namnhuantieptheouwu:
    .string "Nam nhuan ke tiep: "
newline:
    .byte 10
ans:
    .int 0

.section .bss
    .lcomm number1, 5
    buffer_num: .space 16

.section .text
```

```
    .globl _start

jmp _start

namnhuantieptheo:
    addl $1, ans         # Tang nam len 1
    movl ans, %eax
    xorl %edx, %edx
    movl $4, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namnhuantieptheo # Khong chia het cho 4 -> lap tiep

    movl ans, %eax
    xorl %edx, %edx
    movl $100, %ecx
    divl %ecx
    cmpl $0, %edx
    jne okroido          # Chia het cho 4 va khong chia het cho 100 -> Nam nhuan!

    movl ans, %eax
    xorl %edx, %edx
    movl $400, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namnhuantieptheo # Chia het cho 100 nhung khong chia het cho 400 -> Lap tiep

okroido:
    movl $4, %eax
    movl $1, %ebx
    movl $namnhuantieptheouwu, %ecx
    movl $19, %edx
    int $0x80

    movl ans, %eax
    movl $buffer_num + 10, %esi
    movl $0, %ecx

chuyen_doi:
    decl %esi
    xorl %edx, %edx
    movl $10, %ebx
    divl %ebx
    addb $48, %dl
    movb %dl, (%esi)
    incl %ecx
    cmpl $0, %eax
    jne chuyen_doi

    movl $4, %eax
    movl $1, %ebx
    movl %esi, %ecx
    movl $buffer_num + 10, %edx
    subl %esi, %edx
    int $0x80

    call xuonghang
    ret

xuonghang:
    movl $4, %eax
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret

_start:
    movl $4, %eax
    movl $1, %ebx
    movl $nhapvao, %ecx
    movl $23, %edx
    int $0x80

    movl $3, %eax
    movl $0, %ebx
    movl $number1, %ecx
    movl $5, %edx
    int $0x80

    # Chuyen 4 ky tu thanh so nguyen ans
    movzbl number1, %eax
    subl $48, %eax
    imull $1000, %eax
    addl %eax, ans

    movzbl number1+1, %eax
    subl $48, %eax
    imull $100, %eax
    addl %eax, ans

    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans

    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

    # Kiem tra 2 chu so cuoi co cung bang 0 hay khong
    movl $0, %ecx
    addl %eax, %ecx
    addl %ebx, %ecx
    cmpl $0, %ecx
    je tieptuckiemtra
    jmp saudo

tieptuckiemtra:          # Nam the ky: kiem tra chia het cho 400
    movl ans, %eax
    xorl %edx, %edx
    movl $400, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namkhongnhuan

lanamnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamnhuan, %ecx
    movl $10, %edx
    int $0x80
    jmp end

namkhongnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamkhongnhuan, %ecx
    movl $16, %edx
    int $0x80
    jmp end

saudo:                   # Nam binh thuong: kiem tra 2 chu so cuoi chia het cho 4
    movl $0, ans
    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans

    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

    movl ans, %eax
    xorl %edx, %edx
    movl $4, %ecx
    divl %ecx
    cmpl $0, %edx
    jne tnamkhongnhuan

tlanamnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamnhuan, %ecx
    movl $10, %edx
    int $0x80
    jmp end

tnamkhongnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamkhongnhuan, %ecx
    movl $16, %edx
    int $0x80

end:
    # Tinh lai gia tri nam goc va tim nam nhuan tiep theo
    movl $0, ans
    movzbl number1, %eax
    subl $48, %eax
    imull $1000, %eax
    addl %eax, ans
    movzbl number1+1, %eax
    subl $48, %eax
    imull $100, %eax
    addl %eax, ans
    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans
    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

    call namnhuantieptheo

    mov $1, %eax
    int $0x80
```

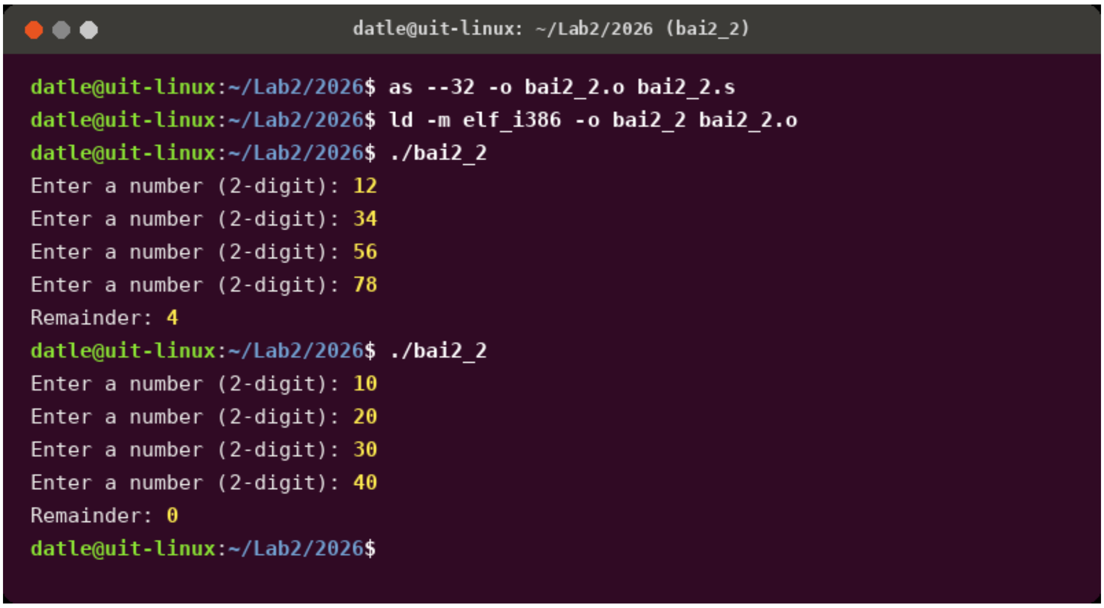
2 Minh chứng kết quả thực thi

Dưới đây là hình ảnh ghi nhận chi tiết quá trình biên dịch với `as --32`, liên kết với `ld -m elf_i386` và chạy kiểm thử thực tế trên Linux Terminal cho toàn bộ 6 bài tập (bao gồm các ca kiểm thử cơ bản, nâng cao và các trường hợp biên):



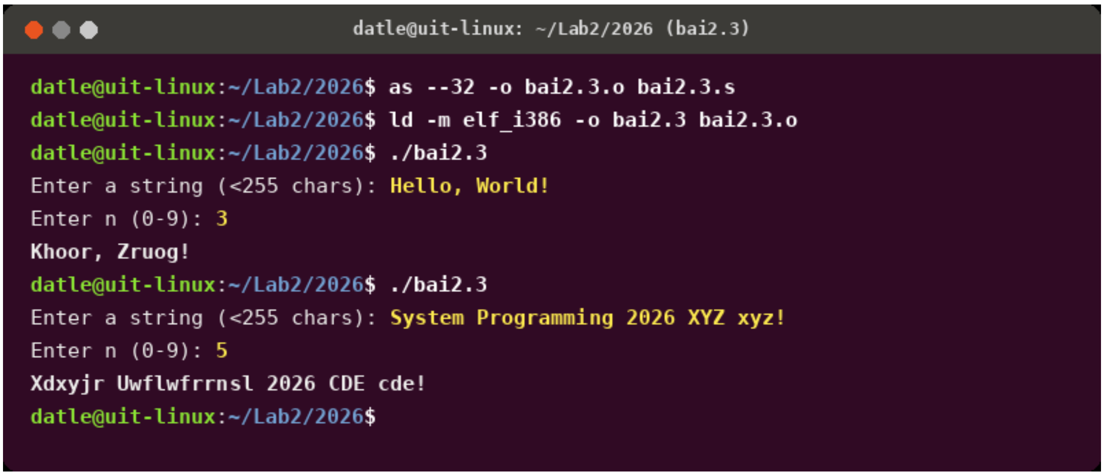
```
datle@uit-linux:~/Lab2/2026$ as --32 -o bai2.1.o bai2.1.s
datle@uit-linux:~/Lab2/2026$ ld -m elf_i386 -o bai2.1 bai2.1.o
datle@uit-linux:~/Lab2/2026$ ./bai2.1
8
datle@uit-linux:~/Lab2/2026$
```

Hình 1: Kết quả biên dịch, liên kết và thực thi Bài 2.1 (In độ dài chuỗi cho trước).



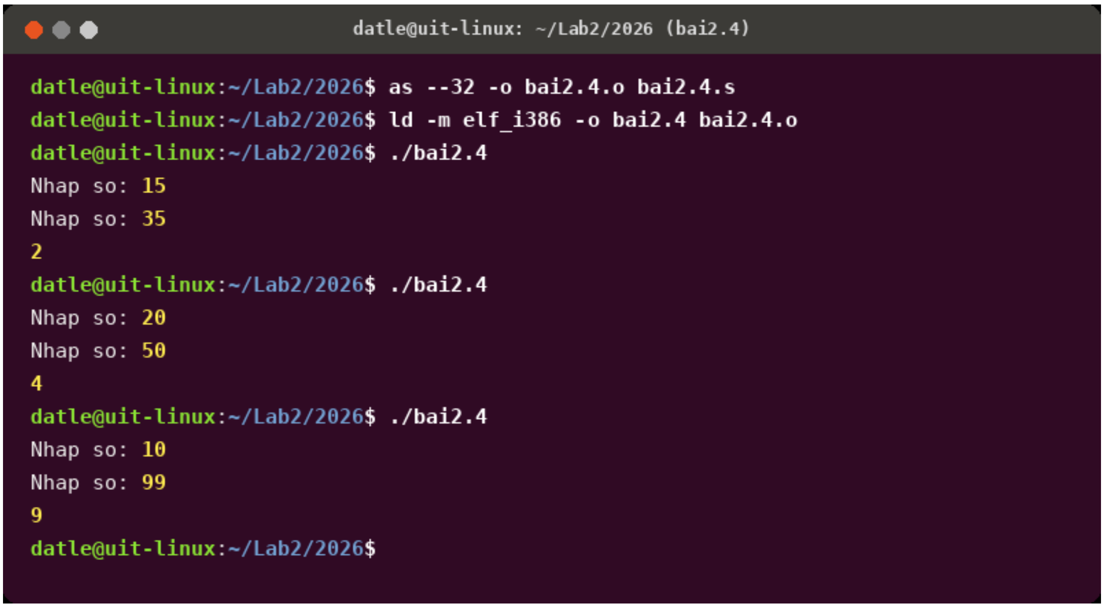
```
datle@uit-linux:~/Lab2/2026$ as --32 -o bai2_2.o bai2_2.s
datle@uit-linux:~/Lab2/2026$ ld -m elf_i386 -o bai2_2 bai2_2.o
datle@uit-linux:~/Lab2/2026$ ./bai2_2
Enter a number (2-digit): 12
Enter a number (2-digit): 34
Enter a number (2-digit): 56
Enter a number (2-digit): 78
Remainder: 4
datle@uit-linux:~/Lab2/2026$ ./bai2_2
Enter a number (2-digit): 10
Enter a number (2-digit): 20
Enter a number (2-digit): 30
Enter a number (2-digit): 40
Remainder: 0
datle@uit-linux:~/Lab2/2026$
```

Hình 2: Kết quả biên dịch, liên kết và thực thi Bài 2.2 (Số dư của tổng 4 số khi chia 8).



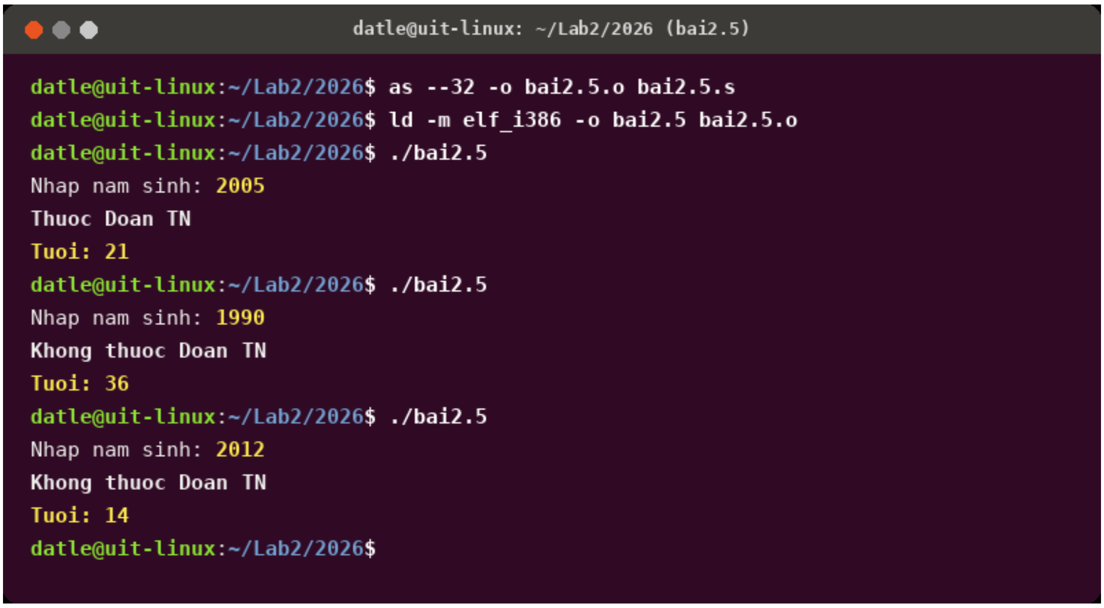
```
datle@uit-linux:~/Lab2/2026$ as --32 -o bai2.3.o bai2.3.s
datle@uit-linux:~/Lab2/2026$ ld -m elf_i386 -o bai2.3 bai2.3.o
datle@uit-linux:~/Lab2/2026$ ./bai2.3
Enter a string (<255 chars): Hello, World!
Enter n (0-9): 3
Khoor, Zruog!
datle@uit-linux:~/Lab2/2026$ ./bai2.3
Enter a string (<255 chars): System Programming 2026 XYZ xyz!
Enter n (0-9): 5
Xdxyjr Uwflwfrrnsl 2026 CDE cde!
datle@uit-linux:~/Lab2/2026$
```

Hình 3: Kết quả biên dịch, liên kết và thực thi Bài 2.3 (Mã hóa Caesar nâng cao 1 & 2).



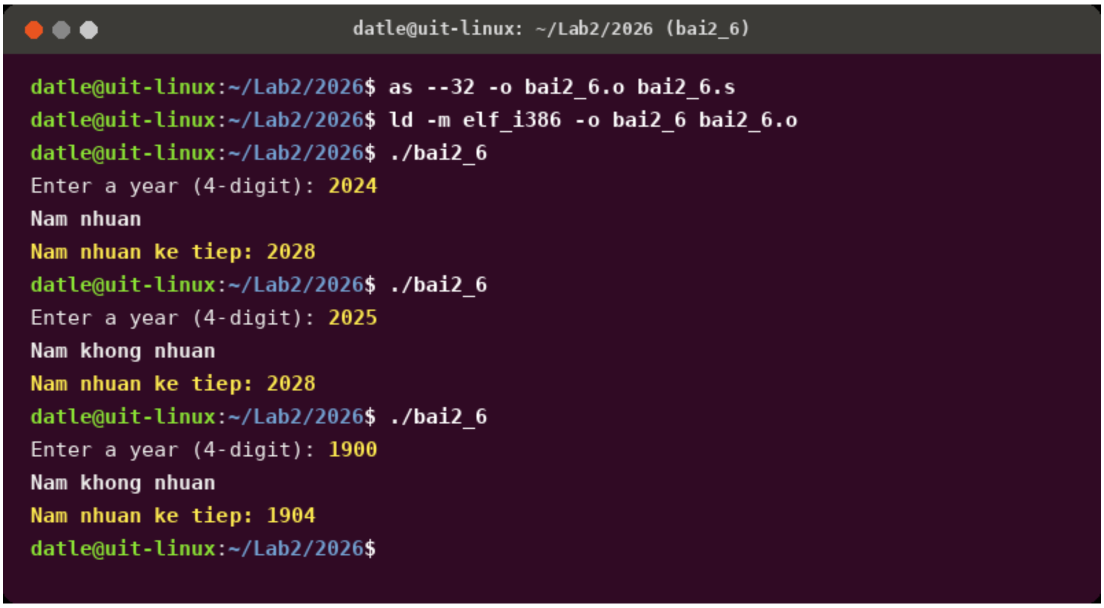
```
datle@uit-linux:~/Lab2/2026$ as --32 -o bai2.4.o bai2.4.s
datle@uit-linux:~/Lab2/2026$ ld -m elf_i386 -o bai2.4 bai2.4.o
datle@uit-linux:~/Lab2/2026$ ./bai2.4
Nhap so: 15
Nhap so: 35
2
datle@uit-linux:~/Lab2/2026$ ./bai2.4
Nhap so: 20
Nhap so: 50
4
datle@uit-linux:~/Lab2/2026$ ./bai2.4
Nhap so: 10
Nhap so: 99
9
datle@uit-linux:~/Lab2/2026$
```

Hình 4: Kết quả biên dịch, liên kết và thực thi Bài 2.4 (Đếm số lượng bội của 10 trong đoạn $[x, y]$).



```
datle@uit-linux: ~/Lab2/2026 (bai2.5)
datle@uit-linux:~/Lab2/2026$ as --32 -o bai2.5.o bai2.5.s
datle@uit-linux:~/Lab2/2026$ ld -m elf_i386 -o bai2.5 bai2.5.o
datle@uit-linux:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 2005
Thuoc Doan TN
Tuoi: 21
datle@uit-linux:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 1990
Khong thuoc Doan TN
Tuoi: 36
datle@uit-linux:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 2012
Khong thuoc Doan TN
Tuoi: 14
datle@uit-linux:~/Lab2/2026$
```

Hình 5: Kết quả biên dịch, liên kết và thực thi Bài 2.5 (Kiểm tra tuổi đoàn và in kèm số tuổi).



```
datle@uit-linux: ~/Lab2/2026 (bai2_6)
datle@uit-linux:~/Lab2/2026$ as --32 -o bai2_6.o bai2_6.s
datle@uit-linux:~/Lab2/2026$ ld -m elf_i386 -o bai2_6 bai2_6.o
datle@uit-linux:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2024
Nam nhuan
Nam nhuan ke tiep: 2028
datle@uit-linux:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2025
Nam khong nhuan
Nam nhuan ke tiep: 2028
datle@uit-linux:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 1900
Nam khong nhuan
Nam nhuan ke tiep: 1904
datle@uit-linux:~/Lab2/2026$
```

Hình 6: Kết quả biên dịch, liên kết và thực thi Bài 2.6 (Kiểm tra năm nhuận và tìm năm nhuận kế tiếp).